SC1001: Introduction to Computational Thinking

Week 10: Recursion on Binary Trees

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

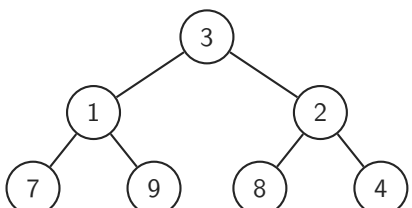
`boydgraber.org/teaching/SC_1001/`

Overview

1. Start from recursive algorithms for complete binary trees
2. Extend the representation to missing children
3. Add the empty-tree base case
4. Reuse the same recursive decomposition

Complete Binary Trees

A complete binary tree representation uses a list with three parts.

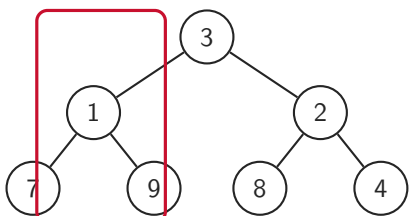



```
tree = [[[7], 1, [9]], 3, [[8], 2, [4]]]
```

Each internal node uses `[left, root, right]`.

Complete Binary Trees

A complete binary tree representation uses a list with three parts.

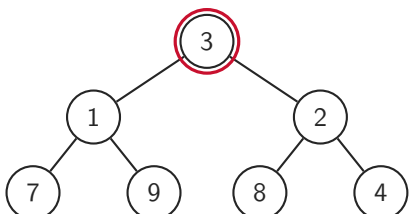



```
tree = [ [[7], 1, [9]] , 3, [[8], 2, [4]]]
```

The first element is the left subtree rooted at `1`.

Complete Binary Trees

A complete binary tree representation uses a list with three parts.

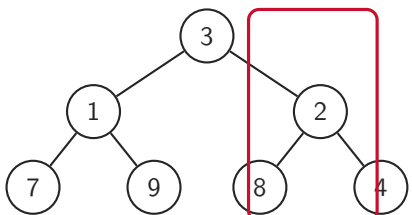



```
tree = [[[7], 1, [9]], 3, [[8], 2, [4]]]
```

The middle element is the root value 3.

Complete Binary Trees

A complete binary tree representation uses a list with three parts.




```
tree = [[[7], 1, [9]], 3, [[8], 2, [4]] ]
```

The last element is the right subtree rooted at 2.

Type Hints

```
from typing import TypeAlias

Tree: TypeAlias = list[] | list[int] | list["Tree", int, "Tree"]

# Tree values must have one of these shapes:
# []                  empty tree
# [root]              leaf node
# [left, root, right] internal node
```

- Function names use `snake_case`.
- Internal nodes always place an `int` between two trees.
- The docstring says what the function returns.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

The function receives one tree and returns an integer.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

The docstring describes the result.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

A complete-tree leaf has length 1.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

A leaf contains exactly one node.

Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Unpack the internal node into its three parts.

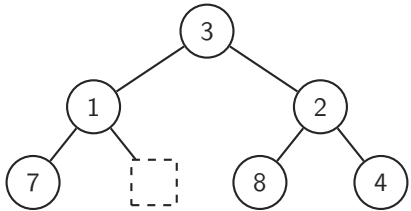
Reference: count_nodes

```
def count_nodes(tree: Tree) -> int:
    """Returns the number of nodes in tree."""
    if len(tree) == 1:
        return 1
    left, root, right = tree
    return count_nodes(left) + count_nodes(right) + 1
```

Count the left subtree, right subtree, and root.

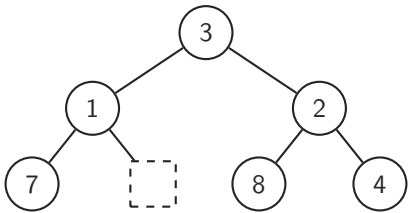
Discussion 1: Binary Tree Representation

How could we extend the list representation to binary trees where a parent node may have one child or two children?



Discussion 2: Count Nodes

How do we extend `count_nodes(tree)` to allow empty children?



Discussion 3: Sum Nodes

How do we extend `sum_nodes(tree)` to allow empty children?

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

The function receives one tree and returns an integer.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

The docstring gives the goal.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

A leaf is the base case.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

For a leaf, the only value is stored at index 0.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Unpack the internal node into left subtree, root, and right subtree.

Reference: sum_nodes

```
def sum_nodes(tree: Tree) -> int:
    """Returns the sum of all node values in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return sum_nodes(left) + root + sum_nodes(right)
```

Add the left sum, root value, and right sum.

Discussion 4: Maximum Node

How do we extend `max_node(tree)` to allow empty children?

What should an empty tree return so it never wins the maximum comparison?

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

The function receives a tree and returns the largest value.

Reference: `max_node`

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

The docstring states the comparison we need.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

A leaf is the base case.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

For a leaf, return its single value.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

Unpack the internal node.

Reference: max_node

```
def max_node(tree: Tree) -> int | float:
    """Returns the largest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return max(max_node(left), root, max_node(right))
```

The maximum is the best among left subtree, root, and right subtree.

Why `-inf`?

Python integers can grow beyond the fixed-width limits used in many other languages.

```
from math import inf

-inf
inf
```

- In C or Java, a smallest integer sentinel can be reasonable
  - A fixed width sequence of bits
  - Can result in overflow
- Python does not have one smallest possible integer: uses as many bits as needed
- IEEE floating point defines infinity values for comparisons.
- `-inf` loses every greater-than comparison with real node values.

Discussion 5: Minimum Node

How do we extend `min_node(tree)` to allow empty children?

What should an empty tree return so it never wins the minimum comparison?

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

The function receives a tree and returns the smallest value.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

The docstring states the comparison we need.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

A leaf is the base case.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

For a leaf, return its single value.

Reference: `min_node`

```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

Unpack the internal node.

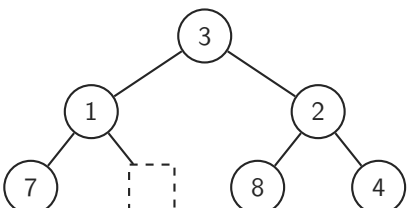
Reference: `min_node`

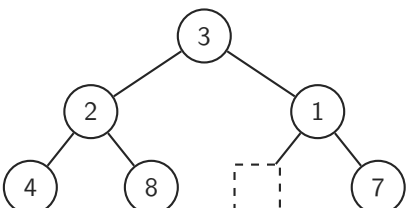
```
def min_node(tree: Tree) -> int | float:
    """Returns the smallest node value in tree."""
    if len(tree) == 1:
        return tree[0]
    left, root, right = tree
    return min(min_node(left), root, min_node(right))
```

The minimum is the best among left subtree, root, and right subtree.

Discussion 6: Mirror

How do we extend `mirror_tree(tree)` to allow empty children?





Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

The function receives a tree and returns a tree.

Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

The docstring names the transformation.

Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

A leaf is already its own mirror.

Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Return the leaf unchanged.

Reference: mirror_tree

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Unpack the three parts of the internal node.

Reference: `mirror_tree`

```
def mirror_tree(tree: Tree) -> Tree:
    """Returns the mirror image of tree."""
    if len(tree) == 1:
        return tree
    left, root, right = tree
    return [mirror_tree(right), root, mirror_tree(left)]
```

Mirror both subtrees, then swap left and right.

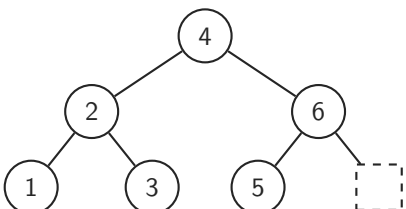
Exercise 1: Maximum Depth

Write `max_depth(tree)`.

- Empty tree depth is `0`.
- A leaf has depth `1`.
- An internal node is one level deeper than its deeper child.

Exercise 2: Binary Search Tree

A binary search tree stores smaller values on the left and larger values on the right.




```
bst = [[[1], 2, [3]], 4, [[5], 6, []]]
```

Write `insert_bst(tree, value)`. If `value` already appears, return the tree unchanged.